Obsidian ERP v4.0 — Architecture Document (Part 2 of 4)

UX Revolution: Guided Flows, Smart Forms & Dashboard Design

> **Product:** Obsidian ERP
> **Company:** VersaLabs Studio
> **Document Version:** 4.0.0
> **Last Updated:** 2026-05-31
> **Depends On:** Part 1 (Foundation)

Table of Contents

---

1. UX Philosophy: From Forms to Flows

1.1 The Core Principle

> **V3 asked: "What data do you want to enter?"**
> **V4 asks: "What do you want to do?"**

Every screen in V4 is designed around **actions, not data entry**. The user expresses intent ("I want to create a sales order"), and the system handles the complexity.

1.2 The UX Shift

Dimension	V3 (Form-First)	V4 (Flow-First)
Create Operation	Full form with 15-20 fields	3-step wizard, 3-5 fields per step, rest auto-filled
List View	Table with filter sidebar	KPI bar + action cards + smart table with inline status changes

Dimension	V3 (Form-First)	V4 (Flow-First)
Detail View	Read-only card layout	Action-oriented dashboard with "What's Next?" prominently displayed
Navigation	Sidebar-only	Sidebar + Command Palette (Cmd+K) + Flow Tracker + breadcrumbs
Status Changes	Hidden in edit form	One-click status buttons on detail page (Submit, Cancel, Amend)
Related Docs	Links in sidebar card	Flow Tracker showing full chain + click-to-create downstream

1.3 The 3-Click Rule

Every common operation in Obsidian ERP must be completable in **3 clicks or fewer**:

```
Click 1: Navigate (sidebar or command palette)
Click 2: Action (Create, Submit, Duplicate)
Click 3: Confirm (Review and save)
```

The SmartForm wizard handles the data — the user just confirms.

1.4 Auto-Population Philosophy

The golden rule of V4: Never ask the user for information the system already knows.

```
Quotation → Sales Order:
  ✅ Customer auto-filled
  ✅ Items auto-filled (quantities, prices, descriptions)
  ✅ Delivery date suggested (today + lead time)
  ✅ Tax template auto-applied (from customer settings)
  ✅ Terms auto-applied (from customer settings)
  🔵 User only confirms: delivery date, any qty changes

Sales Order → Work Order:
  ✅ Production item auto-filled
  ✅ BOM auto-selected (default BOM for item)
  ✅ Quantity auto-filled (from SO line)
  ✅ Warehouse auto-filled (from BOM default)
  🔵 User only confirms: planned start date
```

---

2. The SmartForm Wizard Engine

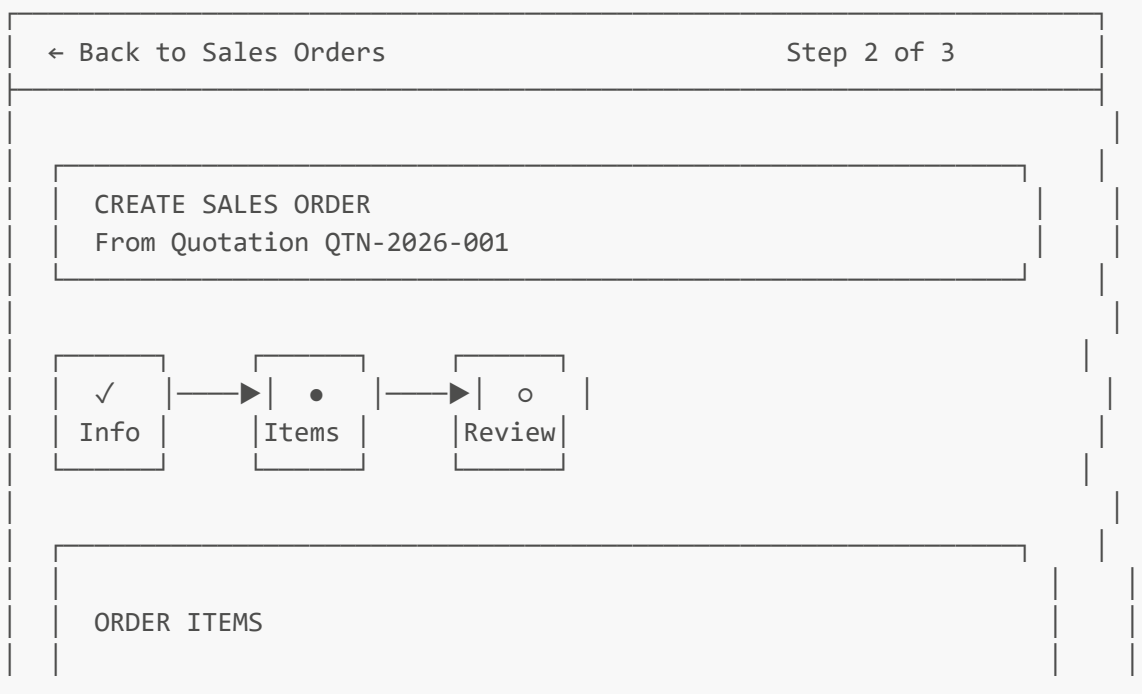
2.1 Architecture

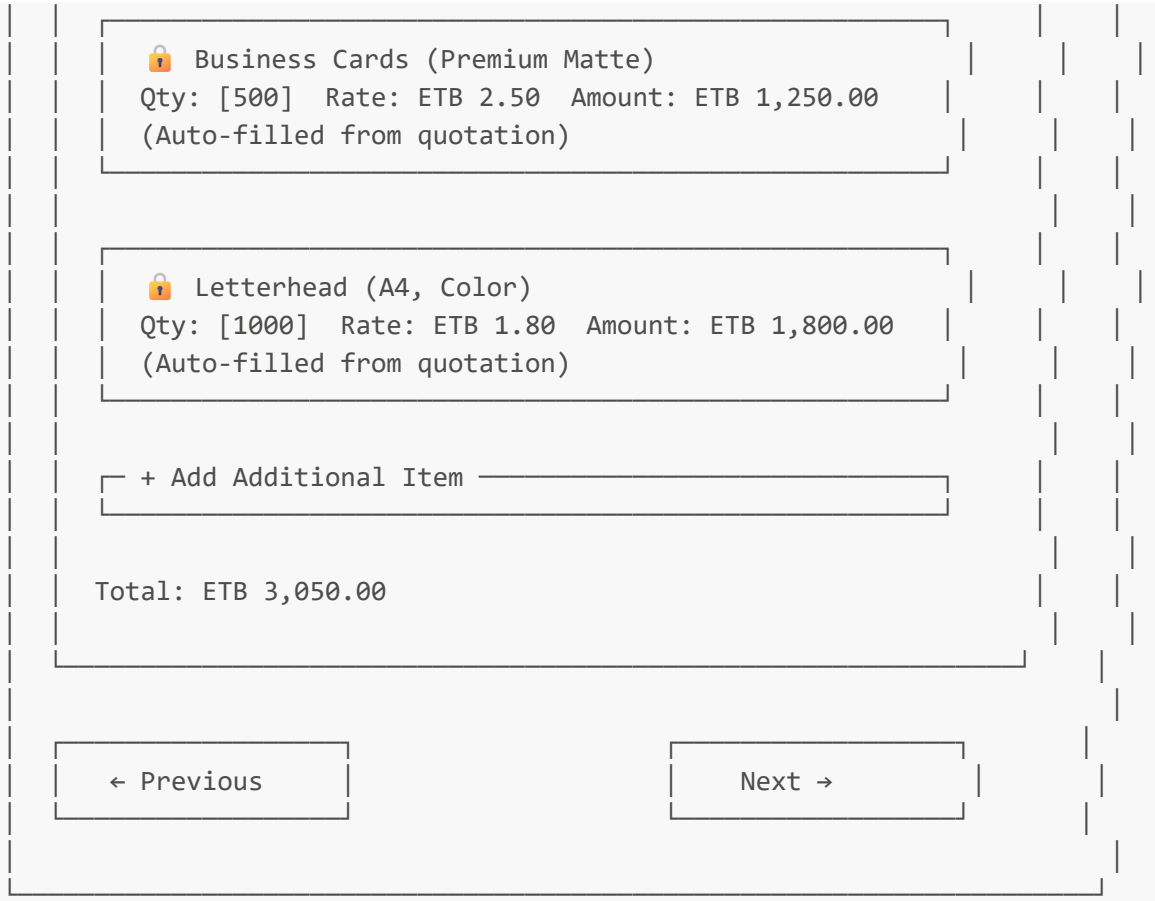
The SmartForm is a reusable wizard component that replaces traditional create/edit forms:

```tsx
// components/flows/FlowWizard.tsx — Core API

interface FlowWizardProps<T extends z.ZodType> {
  definition: FlowDefinition;
  sourceDoc?: Record<string, unknown>;   // Auto-fill source document
  onComplete: (data: z.infer<T>) => void;
  onCancel: () => void;
}

interface FlowDefinition {
  id: string;
  doctype: string;
  title: string;                         // "Create Sales Order"
  subtitle?: string;                     // "From Quotation QTN-2026-001"
  source?: FlowSourceConfig;
  steps: FlowStep[];
  confirmation: FlowConfirmation;
}

interface FlowSourceConfig {
  doctype: string;                       // "Quotation"
  paramName: string;                     // "quotation_id" (URL param)
  fetchEndpoint: string;                 // "/api/sales/quotation/{id}"
  mappings: FieldMapping[];              // How to map source fields to target
}

interface FieldMapping {
  from: string;                          // Source field name
  to: string;                            // Target field name
  transform?: (value: unknown) => unknown; // Optional transformation
}

interface FlowStep {
  id: string;
  title: string;
  description: string;
  icon: LucideIcon;
  fields: FlowField[];
  validationSchema: z.ZodType;
  condition?: (data: Record<string, unknown>) => boolean; // Show step
conditionally
}

interface FlowField {
  name: string;
  label: string;
  type: 'input' | 'textarea' | 'number' | 'date' | 'select' | 'frappe-select' |
        'switch' | 'currency' | 'file' | 'child-table';
  required: boolean;
  autoFilled?: boolean;                  // If true, value came from source doc
  readOnly?: boolean;                    // Display only, no editing
  helpText?: string;
```

```
  placeholder?: string;
  options?: Array<{ label: string; value: string }>;
  frappeConfig?: {                        // For frappe-select type
    doctype: string;
    labelField: string;
    filters?: Array<[string, string, string]>;
  };
  dependsOn?: {                           // Conditional visibility
    field: string;
    value: unknown;
  };
}

interface FlowConfirmation {
  title: string;                          // "Review Sales Order"
  message?: string;                       // "This will create a Sales Order
and..."
  summaryGroups: SummaryGroup[];          // Grouped field preview
  actions?: FlowAction[];                 // Additional actions (e.g., "Also
create Work Order")
}

interface SummaryGroup {
  title: string;
  fields: Array<{
    label: string;
    field: string;
    format?: 'text' | 'currency' | 'date' | 'number' | 'boolean';
  }>;
}
```

2.2 Wizard UI Layout





2.3 Example: Sales Order Wizard Definition

```
// lib/flows/flow-definitions.ts

export const salesOrderFlow: FlowDefinition = {
  id: 'create-sales-order',
  doctype: 'Sales Order',
  title: 'Create Sales Order',
  source: {
    doctype: 'Quotation',
    paramName: 'from_quotation',
    fetchEndpoint: '/api/sales/quotation',
    mappings: [
      { from: 'party_name', to: 'customer' },
      { from: 'customer_name', to: 'customer_name' },
      { from: 'items', to: 'items' },
      { from: 'taxes', to: 'taxes' },
      { from: 'grand_total', to: 'grand_total' },
      { from: 'net_total', to: 'net_total' },
      { from: 'terms', to: 'terms' },
    ],
  },
  steps: [
    {
      id: 'customer-info',
      title: 'Customer & Dates',
```

```
      description: 'Verify customer and set delivery timeline',
      icon: UserIcon,
      fields: [
        {
          name: 'customer',
          label: 'Customer',
          type: 'frappe-select',
          required: true,
          autoFilled: true,
          frappeConfig: { doctype: 'Customer', labelField: 'customer_name' },
        },
        {
          name: 'transaction_date',
          label: 'Order Date',
          type: 'date',
          required: true,
          autoFilled: true,  // Default: today
        },
        {
          name: 'delivery_date',
          label: 'Expected Delivery',
          type: 'date',
          required: true,
          helpText: 'When should this order be delivered?',
        },
        {
          name: 'po_no',
          label: 'Customer PO Number',
          type: 'input',
          required: false,
          placeholder: 'Optional — customer\'s purchase order reference',
        },
      ],
      validationSchema: z.object({
        customer: z.string().min(1),
        transaction_date: z.string().min(1),
        delivery_date: z.string().min(1),
      }),
    },
    {
      id: 'items',
      title: 'Order Items',
      description: 'Review and adjust items from quotation',
      icon: PackageIcon,
      fields: [
        {
          name: 'items',
          label: 'Items',
          type: 'child-table',
          required: true,
          autoFilled: true,
        },
      ],
      validationSchema: z.object({
```

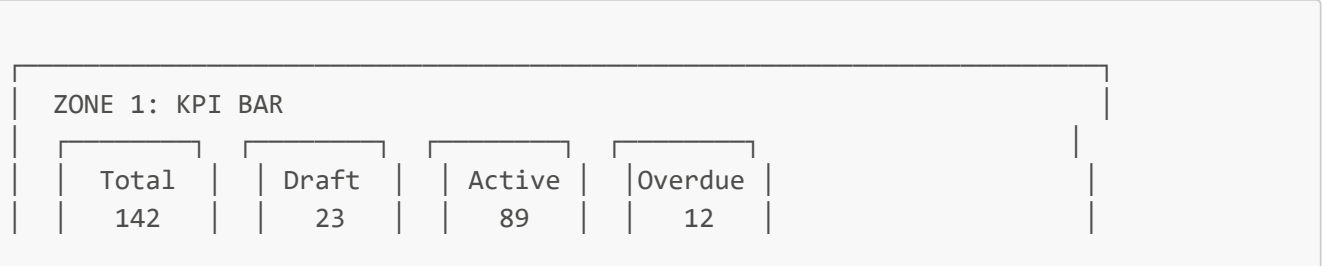
```
        items: z.array(z.object({
          item_code: z.string().min(1),
          qty: z.number().positive(),
          rate: z.number().nonnegative(),
        })).min(1),
      }),
    },
    {
      id: 'review',
      title: 'Review & Confirm',
      description: 'Review your sales order before creating',
      icon: CheckCircleIcon,
      fields: [], // No editable fields — just review
      validationSchema: z.object({}),
    },
  ],
  confirmation: {
    title: 'Create This Sales Order?',
    summaryGroups: [
      {
        title: 'Order Summary',
        fields: [
          { label: 'Customer', field: 'customer_name', format: 'text' },
          { label: 'Order Date', field: 'transaction_date', format: 'date' },
          { label: 'Delivery Date', field: 'delivery_date', format: 'date' },
          { label: 'Total', field: 'grand_total', format: 'currency' },
        ],
      },
    ],
    actions: [
      {
        label: 'Also create Work Order(s)',
        description: 'Automatically create work orders for stock items',
        default: true,
      },
    ],
  },
};
```

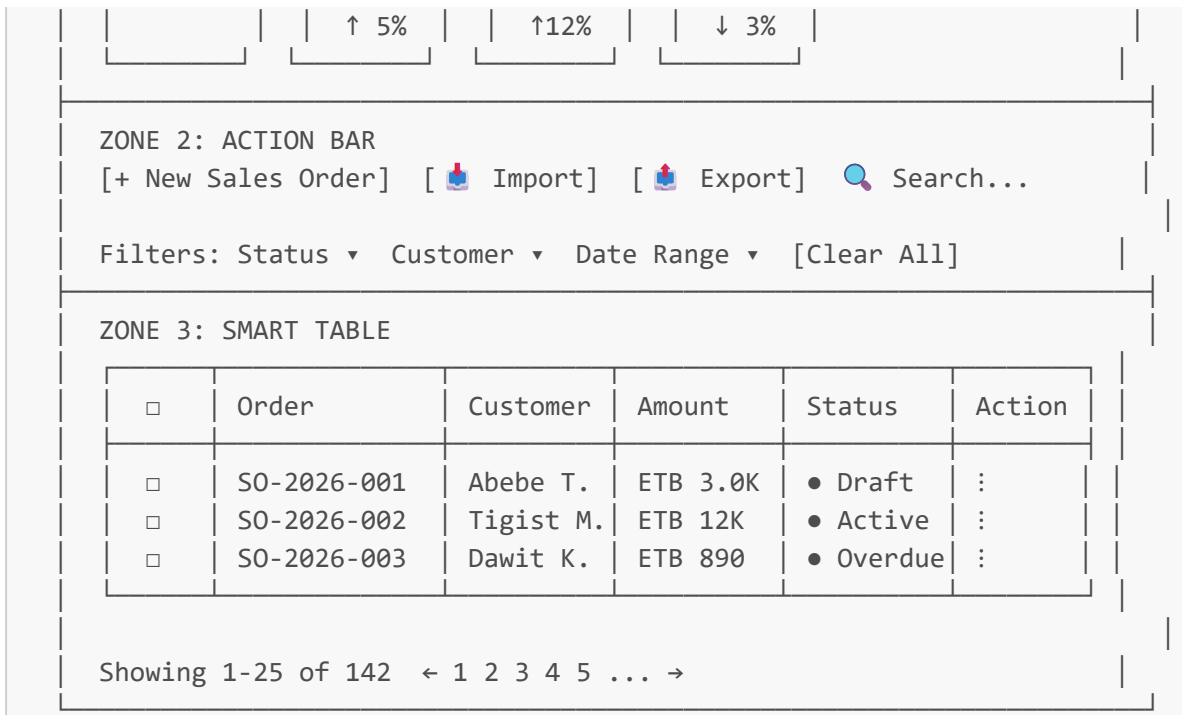
---

3. Page Template Patterns (V4)

3.1 List Page Template

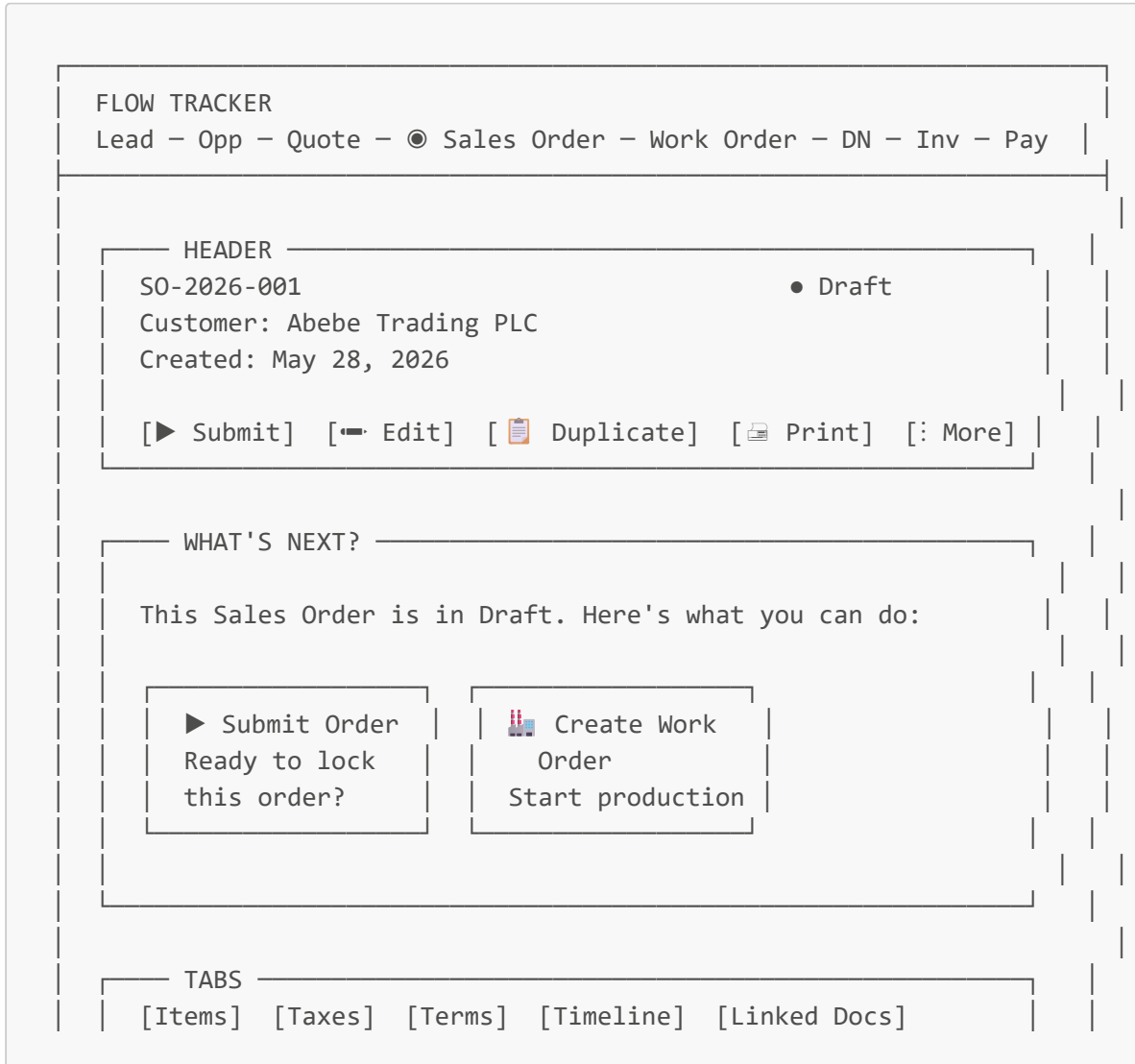
V4 list pages have three zones:

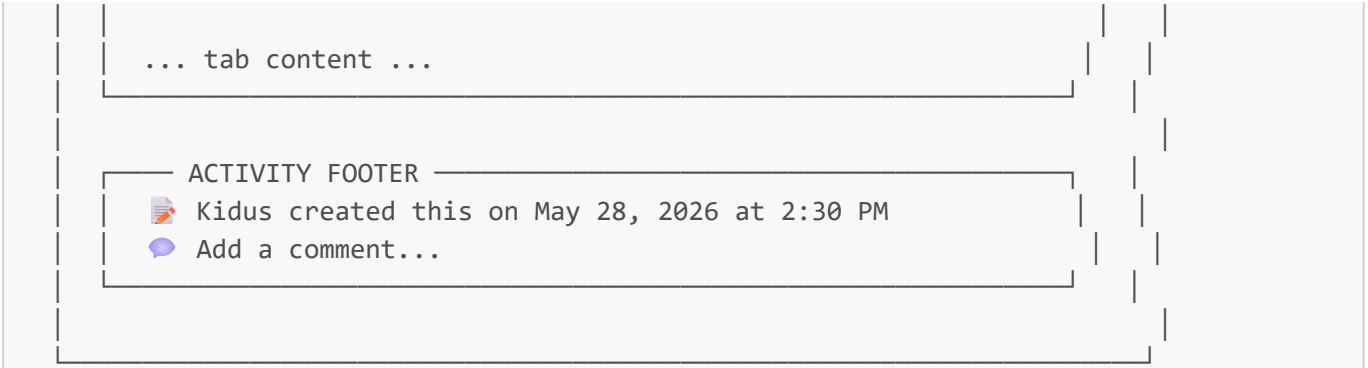




3.2 Detail Page Template

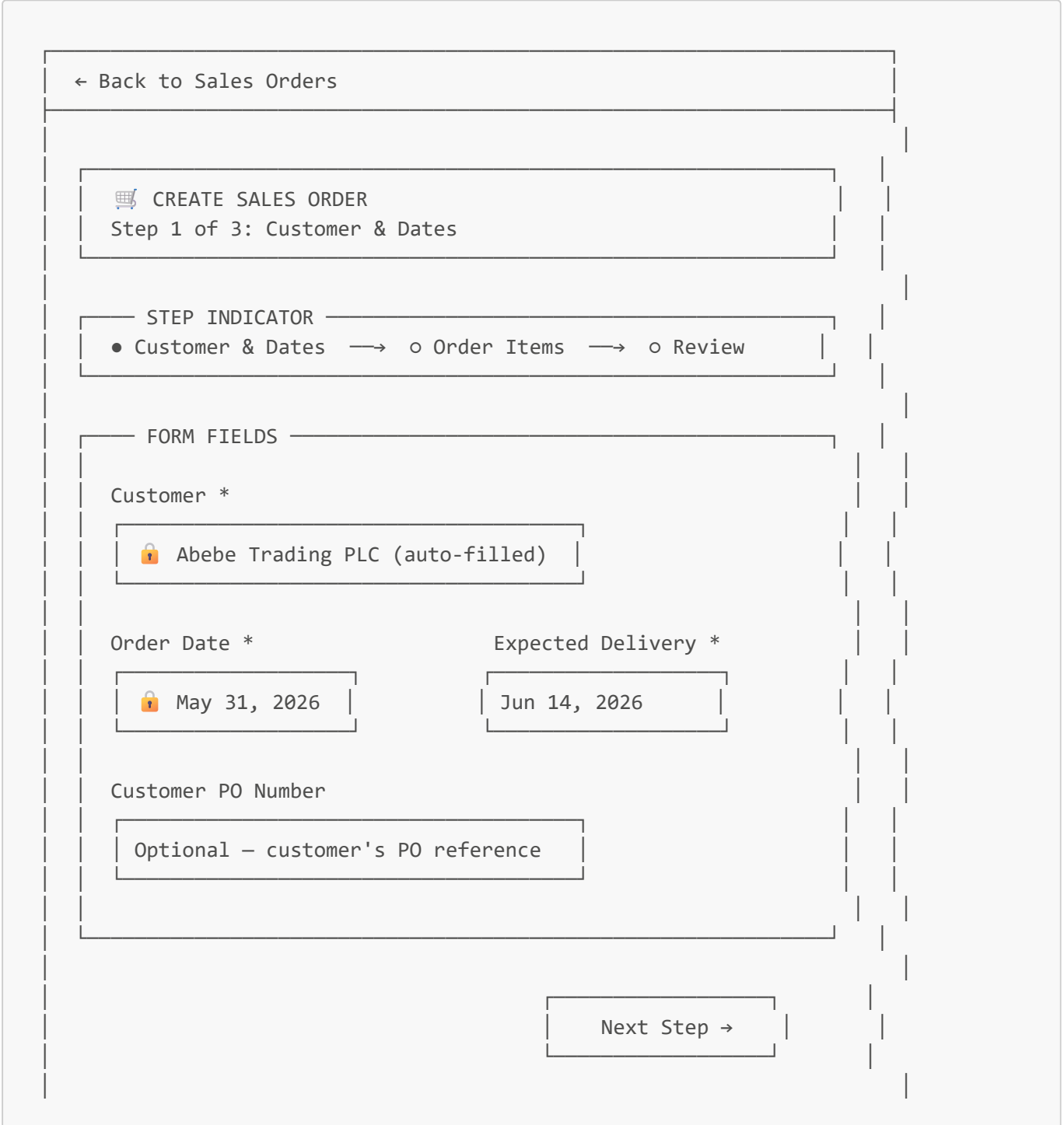
V4 detail pages are **action-oriented**:





3.3 Create Page Template (Wizard)

V4 create pages use the SmartForm Wizard:



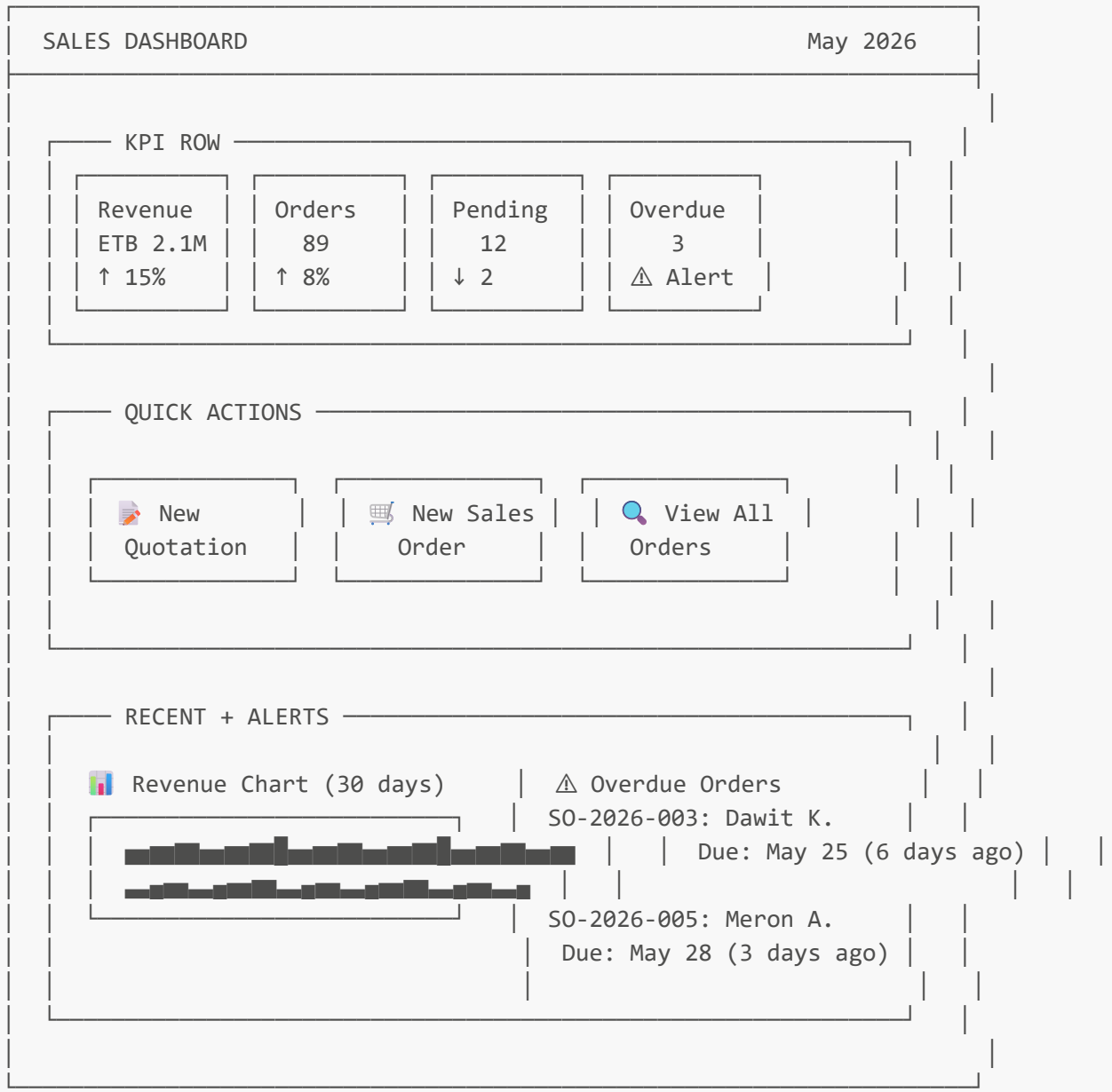
```
└──────────────────────────────────────────────────────────────┘

🔒 = Auto-filled field (read-only, shows source)
```

---

4. Dashboard Design

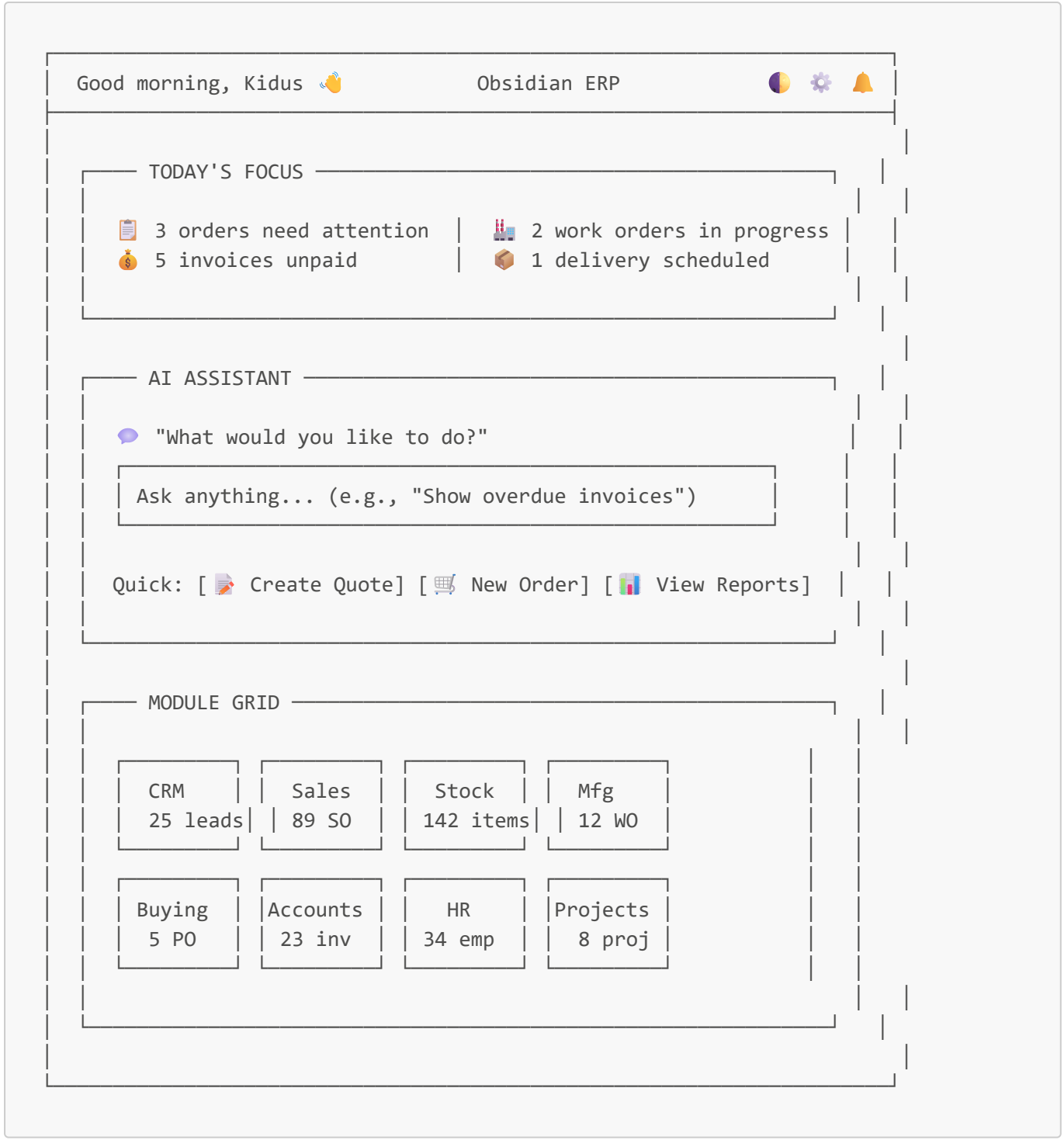
4.1 Module Hub Dashboard

Each module (CRM, Sales, Stock, Manufacturing, Accounting) gets a dashboard:



4.2 Global Home Dashboard

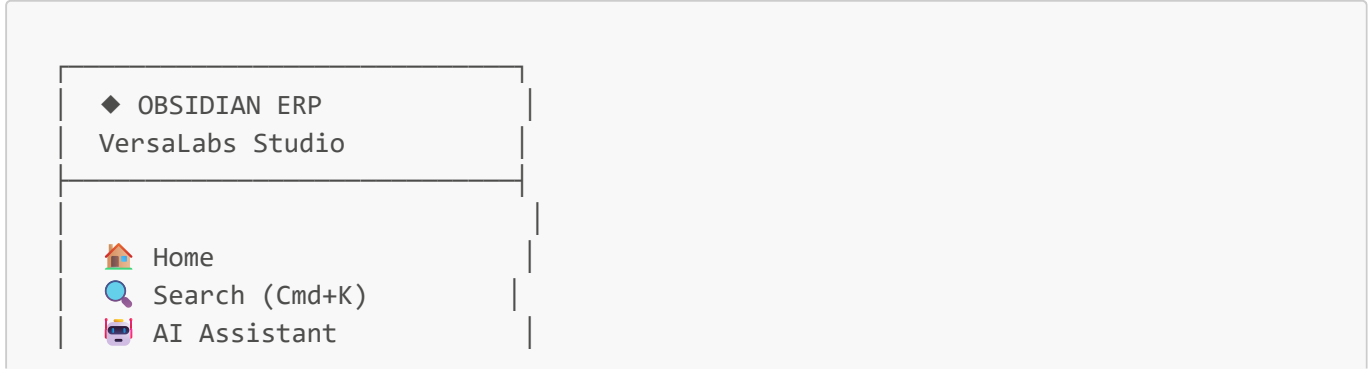
The main landing page shows cross-module overview:

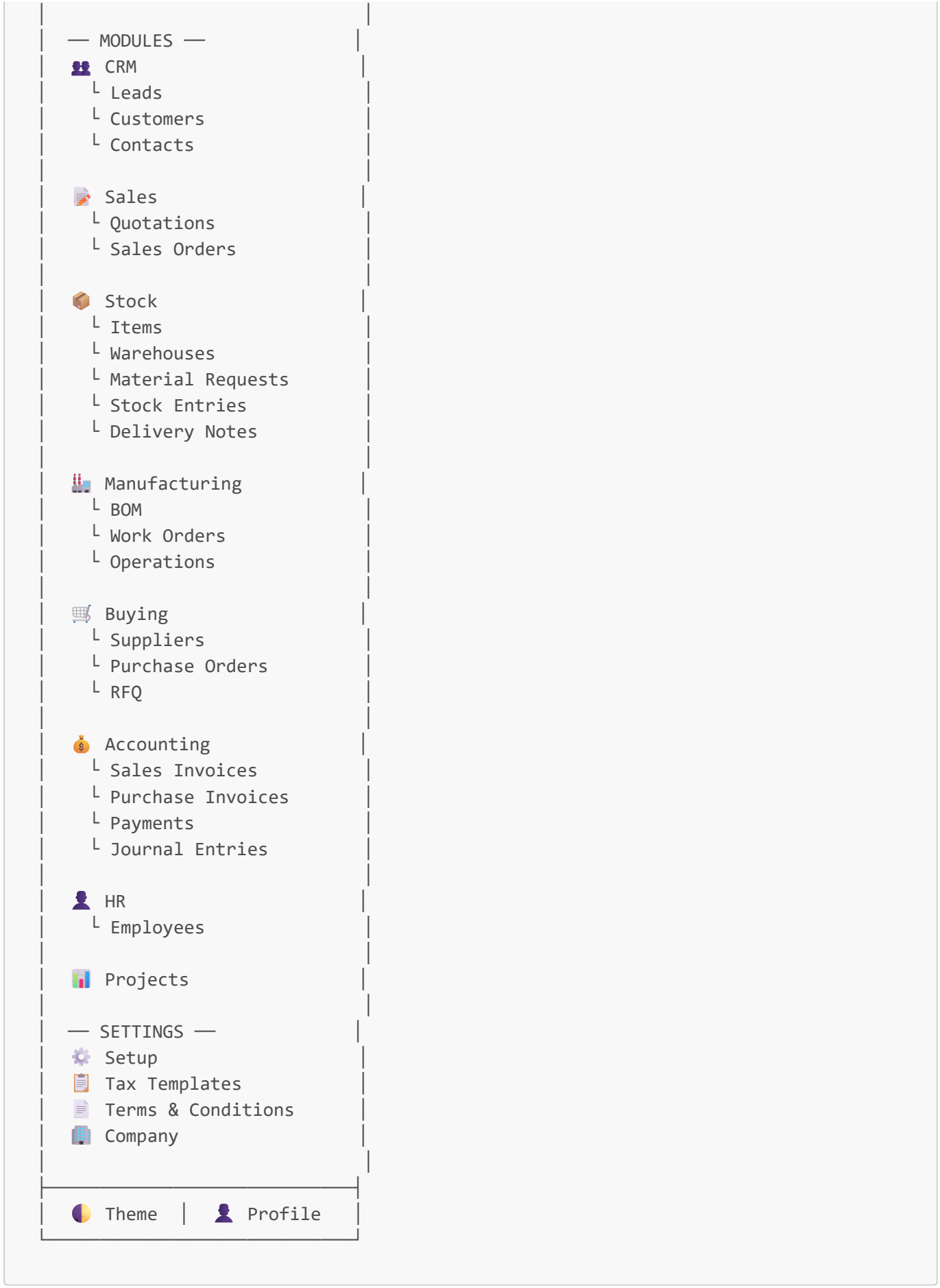


---

5. Navigation System

5.1 Sidebar (Enhanced)





5.2 Command Palette (Cmd+K)

---

6. Flow Tracker Component

6.1 Full Flow Tracker

Appears on every transactional document:

```
// components/flows/FlowTracker.tsx

interface FlowTrackerProps {
  currentDoctype: string;
  currentName: string;
  currentStatus: string;
}

// The flow positions are defined in DocTypeConfigV4.flow.position:
const LEAD_TO_CASH_FLOW = [
  { position: 1, doctype: 'Lead',           label: 'Lead',     icon: UserPlus },
  { position: 2, doctype: 'Opportunity',     label: 'Opp',      icon: Target },
  { position: 3, doctype: 'Quotation',       label: 'Quote',    icon: FileText },
  { position: 4, doctype: 'Sales Order',     label: 'SO',       icon: ShoppingCart
},
```

```
    { position: 5, doctype: 'Work Order',      label: 'WO',       icon: Factory },
    { position: 6, doctype: 'Delivery Note',   label: 'DN',       icon: Truck },
    { position: 7, doctype: 'Sales Invoice',   label: 'Invoice',  icon: Receipt },
    { position: 8, doctype: 'Payment Entry',   label: 'Payment',  icon: CreditCard
  },
  ];
```

6.2 Visual States

```
● Completed (green, clickable — links to actual document)
◉ Current (brand color, pulsing)
○ Pending (gray, shows "Create →" action if next step)
◌ Skipped (dashed, gray)
```

6.3 Smart Context

The Flow Tracker automatically resolves the chain:

```
For SO-2026-001:
  Lead: LEAD-001 (✓ Completed)     ← fetched via lead → opportunity → quotation
chain
  Opp: OPP-001 (✓ Completed)       ← fetched via opportunity → quotation chain
  Quote: QTN-001 (✓ Completed)     ← fetched from quotation field on SO
  SO: SO-2026-001 (◉ Current)      ← this document
  WO: — (○ Create →)               ← not yet created, show action button
  DN: — (○ Pending)
  Invoice: — (○ Pending)
  Payment: — (○ Pending)
```

---

7. Component Library Extensions

7.1 New V4 Components

Component	Location	Purpose
`FlowWizard`	`components/flows/FlowWizard.tsx`	Multi-step wizard engine
`FlowTracker`	`components/flows/FlowTracker.tsx`	Visual workflow progress
`FlowStep`	`components/flows/FlowStep.tsx`	Individual wizard step container
`FlowAutoFill`	`components/flows/FlowAutoFill.tsx`	Auto-population indicator
`KPICard`	`components/dashboard/KPICard.tsx`	Key metric display with trend
`ActionCard`	`components/dashboard/ActionCard.tsx`	Quick action button card

Component	Location	Purpose
FlowStatus	components/dashboard/FlowStatus.tsx	Workflow status summary
CommandPalette	components/command/CommandPalette.tsx	Global search + actions
AICopilot	components/ai/AICopilot.tsx	Main AI interface panel
AIMessage	components/ai/AIMessage.tsx	Chat message bubble
AIActionCard	components/ai/AIActionCard.tsx	Action confirmation card
ActivityTimeline	components/smart/ActivityTimeline.tsx	Activity log display
WhatsNext	components/smart/WhatsNext.tsx	Next action suggestions
SmartTable	components/smart/SmartTable.tsx	Enhanced table with inline actions
InlineStatusChange	components/smart/InlineStatusChange.tsx	One-click status updates

7.2 V3 Components Preserved

All V3 components remain unchanged:

Component	Status	V4 Notes
PageHeader	✅ Preserved	Minor enhancements (add command palette trigger)
FrappeSelect	✅ Preserved	No changes
SearchableSelect	✅ Preserved	No changes
DataField / DataPoint	✅ Preserved	No changes
ConfirmDialog	✅ Preserved	No changes
PrintLabelDialog	✅ Preserved	No changes
EmptyState	✅ Preserved	Enhanced with action suggestions
LoadingState	✅ Preserved	No changes
StatusBadge	✅ Preserved	No changes
ThemeToggle	✅ Preserved	No changes
FormInput	✅ Preserved	No changes
FormTextarea	✅ Preserved	No changes
FormSelect	✅ Preserved	No changes
FormSwitch	✅ Preserved	No changes
FormFrappeSelect	✅ Preserved	No changes

8. V4 Golden Template

8.1 New Golden Template: Sales Order Module

The Sales Order module becomes the V4 Golden Template because it demonstrates:

1. **Wizard flow** (from Quotation to SO)
2. **Flow Tracker** (mid-flow position)
3. **Auto-population** (items, customer, taxes from quotation)
4. **What's Next** (Create Work Order, Submit)
5. **KPI Dashboard** (revenue, count, overdue)
6. **Status management** (Draft → Submitted → Cancelled)
7. **Downstream creation** (SO → Work Order → Stock Entry)
8. **Activity timeline** (comments, status changes)

8.2 File Structure for Golden Template

```
app/sales/sales-order/
├── page.tsx                    # List page (V4: KPI bar + smart table)
├── new/
│   └── page.tsx                # Create page (V4: FlowWizard)
└── [name]/
    ├── page.tsx                # Detail page (V4: FlowTracker + WhatsNext)
    └── edit/
        └── page.tsx            # Edit page (V4: FlowWizard in edit mode)
```

8.3 All New Modules Must Follow This Template

Every module built after the Sales Order Golden Template must:

1. ✅ Use `FlowWizard` for create/edit pages
2. ✅ Include `FlowTracker` on detail pages (if transactional)
3. ✅ Include `WhatsNext` on detail pages
4. ✅ Include `KPICard` bar on list pages
5. ✅ Include `ActionCard` quick actions on list pages
6. ✅ Include `ActivityTimeline` on detail pages
7. ✅ Use `SmartTable` with inline status changes
8. ✅ Auto-fill from upstream documents where applicable
9. ✅ Support Cmd+K command palette actions
10. ✅ Work in both light and dark themes

---

9. Responsive & Mobile Strategy

9.1 Breakpoints

Breakpoint	Width	Layout
Mobile	< 640px	Full-width, stacked, bottom nav
Tablet	640-1024px	Collapsible sidebar, 2-column

Breakpoint	Width	Layout
Desktop	> 1024px	Full sidebar, 3-column

9.2 Mobile-First Principles

1. **Touch targets:** Minimum 44px × 44px for all interactive elements
2. **Wizard steps:** Full-screen steps on mobile (swipe between)
3. **Tables:** Horizontal scroll with sticky first column
4. **Navigation:** Bottom tab bar replaces sidebar on mobile
5. **AI panel:** Full-screen overlay on mobile
6. **Flow Tracker:** Horizontal scroll with current step centered

---

10. Accessibility & Localization

10.1 Accessibility (WCAG AA)

Requirement	Implementation
Color contrast	OKLCH tokens validated for 4.5:1 minimum
Keyboard navigation	All actions accessible via keyboard
Screen reader	Proper `aria-label`, `role`, semantic HTML
Focus indicators	Visible focus rings on all interactive elements
Error announcements	`aria-live` regions for form errors and toasts

10.2 Localization Preparation

V4 prepares for localization but does not implement it in the first release:

Language	Status	Priority
English	✅ Default	GA Release
Amharic (አማርኛ)	🔮 Planned	Post-GA
Arabic (العربية)	🔮 Planned	RTL support needed

Preparation:

- All UI strings extracted to locale files (future)
- RTL-aware layouts using `dir` attribute (future)
- Date/currency formatting using Intl API (current)

Summary

Part 2 establishes:

1. ✅ **UX Philosophy** — from forms to flows, 3-click rule, auto-population
2. ✅ **SmartForm Wizard** — full API specification with type definitions
3. ✅ **Page Templates** — list, detail, create patterns with V4 enhancements
4. ✅ **Dashboard Design** — module dashboards and global home
5. ✅ **Navigation** — sidebar + command palette + flow tracker
6. ✅ **Flow Tracker** — visual workflow progress with smart context
7. ✅ **Component Library** — 15 new components, all V3 preserved
8. ✅ **V4 Golden Template** — Sales Order module as reference
9. ✅ **Responsive strategy** — mobile-first with touch targets
10. ✅ **Accessibility** — WCAG AA compliance plan

Next: Part 3 covers the AI Integration — model selection, tool calling, action execution, and the NL→System Operations pipeline.

Obsidian ERP v4.0 Architecture — Part 2 of 4